Foreman/Katello Hands-On Lab Guide (No Hammer)

Foreman/Katello Hands-On Lab Guide

Homelab: satellite.homelab.local (192.168.50.3) — RHEL 9 (containerized Foreman/Katello)

No `hammer` CLI needed anywhere in this guide. Every task gives you two ways in: - **GUI** — click through the Satellite web interface - **Shell (`curl` + API)** — direct HTTPS calls to Satellite's REST API, run straight from your host shell

The API path is exactly what `hammer` would call under the hood anyway, so you're not missing anything real-world by skipping it.

One-Time Shell Setup

Run this once per terminal session (or add to `~/.bashrc`) so every command below just works:

```
export SAT_USER=admin
export SAT_PASS='yourpass'
export SAT_URL=https://satellite.homelab.local

# Quick connectivity test
curl -k -u $SAT_USER:$SAT_PASS $SAT_URL/api/v2/ping
```

If that returns JSON with a status, you're good for everything below. (`-k` skips SSL verification since your cert is self-signed — fine for homelab use.)

Handy helper — extract an `id` field from a JSON response without needing `jq` installed:

```
# Usage: get_id "<json_response>"
get_id() { python3 -c "import sys,json;print(json.loads(sys.argv[1])['results'][0]['id'])" "$1"; }
```

Task 1: Organization & Location Structure

GUI: 1. Administer → Organizations → New Organization → name: `HomelabCorp` → Submit 2. Administer → Locations → New Location → name: `DataCenter-Hyd` → Submit 3. Repeat for `DataCenter-Blr` 4. Assign your admin user to both under Administer → Users → Edit → Organizations/Locations tabs

Shell:

```
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/api/v2/organizations \
  -d '{"organization": {"name": "HomelabCorp", "label": "homelabcorp"}}'

curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/api/v2/locations \
  -d '{"location": {"name": "DataCenter-Hyd"}}'
```

Verify:

```
curl -k -u $SAT_USER:$SAT_PASS $SAT_URL/api/v2/organizations | python3 -m json.tool
```

Save the Org ID for later tasks:

```
ORG_ID=$(curl -sk -u $SAT_USER:$SAT_PASS $SAT_URL/api/v2/organizations | python3 -c "import
sys,json;print(json.load(sys.stdin)['results'][0]['id'])")
echo $ORG_ID
```

Real-world tie-in: Every enterprise Satellite deployment starts here. This is the tenancy boundary — nothing else (repos, hosts, content views) exists without being scoped to an Org first. Banks split by business unit; global companies split by geography.

Task 2: Content Management (Repos & Sync Plans)

GUI: 1. Content → Products → New Product → name: `RockyLinux9` 2. Inside the product → New Repository: - Name: `Rocky9-BaseOS` , Type: `yum` , URL: `https://download.rockylinux.org/pub/rocky/9/BaseOS/x86_64/os/` 3. Repeat for `Rocky9-AppStream` (URL ending in

`/AppStream/x86_64/os/`) 4. Click **Sync Now** on each repo 5. Content → Sync Plans → New Sync Plan → name `Daily-2AM`, interval: daily, start time: 02:00 → attach to your Product

Shell:

```
        # Create product
        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/katello/api/v2/products \
          -d "{\"organization_id\": $ORG_ID, \"name\": \"RockyLinux9\"}"

        PRODUCT_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/katello/api/v2/products?organization_id=$ORG_ID" | python3 -c
"import sys,json;print(json.load(sys.stdin)['results'][0]['id'])")

        # Create repository
        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/katello/api/v2/repositories \
          -d "{\"product_id\": $PRODUCT_ID, \"name\": \"Rocky9-BaseOS\", \"content_type\": \"yum\", \"url\":
\"https://download.rockylinux.org/pub/rocky/9/BaseOS/x86_64/os/\"}"

        REPO_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/katello/api/v2/repositories?product_id=$PRODUCT_ID" | python3 -c
"import sys,json;print(json.load(sys.stdin)['results'][0]['id'])")

        # Trigger sync (async — returns a task ID)
        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/katello/api/v2/repositories/$REPO_ID/sync
```

Verify (sync can take a while — check content counts once done):

```
        curl -k -u $SAT_USER:$SAT_PASS $SAT_URL/katello/api/v2/repositories/$REPO_ID | python3 -m json.tool | grep -A5
content_counts
```

Real-world tie-in: This is what your local RPM repo/Apache experiment was building toward manually — Satellite formalizes and schedules it. Companies mirror upstream once internally for bandwidth control, air-gapped environments, and consistent patch timing across thousands of servers.

Task 3: Content Views & Lifecycle Environments

GUI: 1. Content → Lifecycle Environments → New Environment → `Development` (path: from Library) → then `Testing` (after Development) → then `Production` (after Testing) 2. Content → Content Views → New Content View → name `Rocky9-CV` 3. Add Repositories tab → select `Rocky9-BaseOS`, `Rocky9-AppStream` → Add 4. Publish → version 1.0 5. Promote: Library → Development → Testing → Production (one click each, in order)

Shell:

```
        # Create lifecycle environments (chained — each needs prior_id)
        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/katello/api/v2/environments \
          -d "{\"organization_id\": $ORG_ID, \"name\": \"Development\", \"prior_id\": 1}"

        DEV_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/katello/api/v2/environments?organization_id=$ORG_ID" | python3 -c
"import sys,json;d=json.load(sys.stdin)['results'];print([e['id'] for e in d if e['name']=='Development'][0])")

        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/katello/api/v2/environments \
          -d "{\"organization_id\": $ORG_ID, \"name\": \"Testing\", \"prior_id\": $DEV_ID}"

        TEST_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/katello/api/v2/environments?organization_id=$ORG_ID" | python3 -c
"import sys,json;d=json.load(sys.stdin)['results'];print([e['id'] for e in d if e['name']=='Testing'][0])")

        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/katello/api/v2/environments \
          -d "{\"organization_id\": $ORG_ID, \"name\": \"Production\", \"prior_id\": $TEST_ID}"

        PROD_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/katello/api/v2/environments?organization_id=$ORG_ID" | python3 -c
"import sys,json;d=json.load(sys.stdin)['results'];print([e['id'] for e in d if e['name']=='Production'][0])")

        # Create content view and add the repo
        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/katello/api/v2/content_views \
          -d "{\"organization_id\": $ORG_ID, \"name\": \"Rocky9-CV\"}"

        CV_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/katello/api/v2/content_views?organization_id=$ORG_ID" | python3 -c
"import sys,json;print(json.load(sys.stdin)['results'][0]['id'])")

        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X PUT \
          $SAT_URL/katello/api/v2/content_views/$CV_ID \
          -d "{\"repository_ids\": [$REPO_ID]}"

        # Publish (async — note the task id in the response)
        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/katello/api/v2/content_views/$CV_ID/publish \
          -d '{"description": "Initial publish"}'
```

After publish finishes, get the version ID and promote through each environment:

```
        CV_VERSION_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/katello/api/v2/content_view_versions?content_view_id=$CV_ID
python3 -c "import sys,json;print(json.load(sys.stdin)['results'][0]['id'])")

        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/katello/api/v2/content_view_versions/$CV_VERSION_ID/promote \
          -d "{\"environment_id\": $DEV_ID}"

        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/katello/api/v2/content_view_versions/$CV_VERSION_ID/promote \
          -d "{\"environment_id\": $TEST_ID}"

        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/katello/api/v2/content_view_versions/$CV_VERSION_ID/promote \
          -d "{\"environment_id\": $PROD_ID}"
```

Note: publish/promote are async — each returns a `task` id. Check status with:

```
        curl -k -u $SAT_USER:$SAT_PASS $SAT_URL/foreman_tasks/api/tasks/<task_id> | python3 -m json.tool | grep state
```

Wait until `"state": "stopped"` before moving to the next step.

Real-world tie-in: This is the core reason Satellite exists. You never patch Production directly — you freeze a tested Content View version and promote the *exact same, tested* package set through Dev → Test → Prod. No “worked in staging, broke prod” incidents.

Task 4: Activation Keys

GUI: 1. Content → Activation Keys → New Activation Key - Name: `rocky9-prod-key`, Lifecycle Environment: `Production`, Content View: `Rocky9-CV` 2. Subscriptions tab → attach available subscription (skip if custom repos only)

Shell:

```
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/katello/api/v2/activation_keys \
  -d "{\"organization_id\": $ORG_ID, \"name\": \"rocky9-prod-key\", \"content_view_id\": $CV_ID, \"environment_id\":
$PROD_ID}"
```

Real-world tie-in: When provisioning 50 servers a week, nobody manually configures repos on each one. The provisioning pipeline just passes an activation key and the server auto-configures its entire content/patch relationship — the backbone of zero-touch fleet onboarding.

Task 5: Host Registration

This runs on the client VM, not Satellite — no hammer or Satellite-side CLI involved at all.

RHEL/Rocky client:

```
sudo rpm -Uvh https://satellite.homelab.local/pub/katello-ca-consumer-latest.noarch.rpm
sudo subscription-manager register \
  --org="HomelabCorp" --activationkey="rocky9-prod-key" \
  --serverurl=https://satellite.homelab.local:8443/rhsm
```

Ubuntu client:

```
curl -O https://satellite.homelab.local/pub/bootstrap.py
sudo python3 bootstrap.py --login=admin --organization="HomelabCorp" \
  --activationkey="ubuntu-prod-key" --server satellite.homelab.local
```

Verify (from your shell, hitting Satellite's API):

```
curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/hosts?organization_id=$ORG_ID" | python3 -m json.tool | grep name
```

Real-world tie-in: Real fleets are rarely 100% one OS. Hitting friction registering Ubuntu against a Katello-centric tool is valuable troubleshooting experience — most companies run exactly this kind of mixed RHEL/Ubuntu environment.

Task 6: Host Groups

GUI: 1. Configure → Host Groups → New Host Group - Name: `Web-Servers`, Lifecycle Environment: `Production`, Content View: `Rocky9-CV`, Activation Key: `rocky9-prod-key` 2. Assign an existing host: Hosts → select host → Edit → Host Group: `Web-Servers`

Shell:

```
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
  $SAT_URL/api/v2/hostgroups \
  -d "{\"hostgroup\": {\"name\": \"Web-Servers\", \"organization_ids\": [$ORG_ID]}}"
```

Real-world tie-in: New host provisioning becomes "assign to Web-Servers" — patching, config, and compliance are all inherited automatically. No drift between "identical" servers set up manually on different days.

Task 7: Errata Management (Patch Management)

GUI: 1. Content → Errata — filter by Security/Bugfix/Enhancement or by CVE 2. Hosts → select host → Errata tab → view applicable errata → Apply

Shell:

```
HOST_ID=$(curl -sk -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/hosts?search=name=<hostname>" | python3 -c "import
sys,json;print(json.load(sys.stdin)['results'][0]['id'])")

# List applicable errata
curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/hosts/$HOST_ID/errata" | python3 -m json.tool

# Apply a specific errata
curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X PUT \
  $SAT_URL/api/v2/hosts/bulk/install_content \
  -d "{\"organization_id\": $ORG_ID, \"included\": {\"ids\": [$HOST_ID]}, \"content_type\": \"errata\", \"content\":
[\"RHSA-2026:XXXX\"]}"
```

Verify: Re-run the errata list call — the applied one should no longer appear.

Real-world tie-in: This is literally what a patch management team does monthly — triage which CVEs matter for which servers, patch selectively rather than risk breaking things with a blanket update.

Task 8: Remote Execution (Rex)

GUI: 1. Hosts → select multiple hosts → Select Action → Run Job 2. Job Category: `Commands`, Template: `Run Command - Script Default` 3. Command: `yum update -y` → Schedule: Now

Shell:

```
        # Find the right template ID first
        curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/job_templates" | python3 -c "import sys,json;[print(t['id'],t['name']
t in json.load(sys.stdin)['results']]"

        # Invoke a job (replace <template_id> with the id for 'Run Command - Script Default')
        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/api/v2/job_invocations \
          -d '{"job_invocation": {"job_template_id": <template_id>, "inputs": {"command": "yum update -y"}, "search_query":
"hostgroup = Web-Servers"}}'
```

Verify: Monitor → Jobs in the GUI, or:

```
        curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/job_invocations" | python3 -m json.tool | head -30
```

Real-world tie-in: Fleet-wide command execution without SSH-loop scripting — patch 200 servers or push a config change from one place. Directly ties into your Ansible/automation interests.

Task 9: Compliance (OpenSCAP)

GUI: 1. Hosts → Compliance → Policies → New Compliance Policy - Name: `CIS-Rocky9`, SCAP Content: a CIS/DISA profile, Deployment: Host Group `Web-Servers` 2. Hosts → Compliance → Reports — view pass/fail per rule after a scan runs

Shell (scan runs on the client, not Satellite):

```
        sudo yum install -y openscap-scanner scap-security-guide
        sudo oscap xccdf eval --profile xccdf_org.ssgproject.content_profile_cis \
          --results /tmp/scan-results.xml --report /tmp/scan-report.html \
          /usr/share/xml/scap/ssg/content/ssg-rl9-ds.xml
```

List policies from Satellite's API:

```
        curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/compliance_policies" | python3 -m json.tool
```

Real-world tie-in: Directly builds on your STIG/hardening exploration. Auditors in regulated industries (banking, healthcare) require these reports as compliance evidence.

Task 10: Provisioning Templates (PXE/Kickstart via Satellite)

GUI: 1. Infrastructure → Compute Resources → New Compute Resource → add your ESXi host (Provider: VMware, vCenter credentials) 2. Hosts → New Host: - Host Group: `Web-Servers`, Compute Resource: your ESXi resource - OS: Rocky Linux 9, Kickstart default template 3. Submit — Satellite creates the VM on ESXi, PXE-boots, kicksarts, and auto-registers via the activation key from the host group

Shell (once the compute resource exists in Satellite):

```
        # Get compute resource id
        curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/compute_resources" | python3 -c "import sys,json;[print(c['id'],c['na
for c in json.load(sys.stdin)['results']]"

        # Get hostgroup id
        curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/hostgroups" | python3 -c "import sys,json;[print(h['id'],h['name']) f
in json.load(sys.stdin)['results']]"

        # Create the host (provision)
        curl -k -u $SAT_USER:$SAT_PASS -H "Content-Type: application/json" -X POST \
          $SAT_URL/api/v2/hosts \
          -d "{\"host\": {\"name\": \"web01\", \"hostgroup_id\": <hostgroup_id>, \"compute_resource_id\": <cr_id>,
\"organization_id\": $ORG_ID, \"build\": true}}"
```

Verify: Watch the VM appear in vCenter/ESXi, then:

```
        curl -k -u $SAT_USER:$SAT_PASS "$SAT_URL/api/v2/hosts?search=name=web01" | python3 -m json.tool
```

Real-world tie-in: The highest-value skill in this whole list — full “bare-metal-to-production-ready-server-in-15-minutes” automation, tying Tasks 1–9 together into one flow.

Suggested Order of Attack

Work Tasks **1 → 2 → 3 → 4 → 5 → 7 → 8** first (fastest path to something demoable in an interview), then circle back to **6, 9, 10**.

Every `ORG_ID`, `PRODUCT_ID`, `REPO_ID`, `CV_ID`, etc. variable set in earlier tasks is reused in later ones — keep your terminal session open, or re-run the “get id” lookups at the top of each task if you start a new session.